CSA1702

ARTIFICIAL INTELLIGENCE LAB

Experiments 8–16

Name: Mohammad ALEYAS

Registration No.: 192521220

Course: AI

Course Code: CSA1702

Each experiment includes Aim, Algorithm / Procedure, Python Code, Terminal Output, and Result.

Experiment 8: Depth-First Search (DFS)

Aim: To write a Python program to implement the Depth-First Search (DFS) algorithm for graph traversal.

Algorithm / Procedure

1. Create a graph using an adjacency list.
2. Start from the given node.
3. Mark the node as visited and print it.
4. Visit each unvisited adjacent node recursively.
5. Stop when all reachable nodes are visited.

Code (screenshot-style)

PYTHON CODE

```
def dfs(graph, node, visited):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [], 'E': [], 'F': []
}

visited = set()
print("DFS Traversal:")
dfs(graph, 'A', visited)
```

Output (screenshot-style)

TERMINAL OUTPUT

```
DFS Traversal:\nA B D E C F
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 9: Travelling Salesman Problem (TSP)

Aim: To find the shortest tour that visits each city once and returns to the starting city.

Algorithm / Procedure

1. Represent distances between cities using a matrix.
2. Fix the first city as the starting point.
3. Generate all possible orders of the remaining cities.
4. Calculate the total distance of each tour.
5. Display the tour with the minimum distance.

Code (screenshot-style)

PYTHON CODE

```
from itertools import permutations

dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(dist)
min_cost = float('inf')
best_path = []

for p in permutations(range(1, n)):
    path = (0,) + p + (0,)
    cost = 0
    for i in range(len(path) - 1):
        cost += dist[path[i]][path[i + 1]]
    if cost < min_cost:
        min_cost = cost
        best_path = path

print("Best Tour:", " -> ".join(str(x + 1) for x in best_path))
print("Minimum Cost:", min_cost)
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Best Tour: 1 -> 2 -> 4 -> 3 -> 1\nMinimum Cost: 80
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 10: A* Algorithm

Aim: To implement A* search to find a path from a start node to a goal node using path cost and heuristic values.

Algorithm / Procedure

1. Define the graph with edge costs.
2. Assign heuristic values to each node.
3. Start from the initial node.
4. Calculate f(n) = g(n) + h(n).
5. Expand the node with the lowest f(n).
6. Continue until the goal is reached and display the path.

Code (screenshot-style)

PYTHON CODE

```
import heapq

graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'C': 2, 'D': 5},
    'C': {'D': 1},
    'D': {}
}
h = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def astar(start, goal):
    pq = [(h[start], 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            print("Path:", " -> ".join(path))
            print("Cost:", g)
            return
        if node in visited:
            continue
        visited.add(node)
        for neighbor, cost in graph[node].items():
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + h[neighbor]
                heapq.heappush(
                    pq, (new_f, new_g, neighbor, path + [neighbor])
                )

astar('A', 'D')
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Path: A -> B -> C -> D\nCost: 4
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 11: Map Coloring Using CSP

Aim: To implement Map Coloring using a Constraint Satisfaction Problem (CSP) approach.

Algorithm / Procedure

1. Define regions and neighboring regions.
2. Define the available colors.
3. Assign a color to one region.
4. Check that neighboring regions do not have the same color.
5. If a conflict occurs, try another color using backtracking.
6. Display the valid color assignment.

Code (screenshot-style)

PYTHON CODE

```
regions = ['A', 'B', 'C', 'D']
neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
colors = ['Red', 'Green', 'Blue']
assignment = {}

def is_valid(region, color):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_valid(region, color):
            assignment[region] = color
            if solve(index + 1):
                return True
            del assignment[region]
    return False

if solve(0):
    print("Map Coloring:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution exists")
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Map Coloring:\nA : Red\nB : Green\nC : Blue\nD : Red
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 12: Tic-Tac-Toe Game

Aim: To implement a Tic-Tac-Toe game for two players.

Algorithm / Procedure

1. Create a 3 × 3 board.
2. Allow players X and O to enter positions.
3. Check whether each entered position is valid.
4. Check rows, columns and diagonals for a winner.
5. Repeat until a player wins or the board is full.
6. Display the winner or a draw.

Code (screenshot-style)

PYTHON CODE

```
board = [' '] * 9

def display():
    for i in range(0, 9, 3):
        print('|'.join(board[i:i + 3]))
        if i < 6:
            print("-----")

def check_winner():
    lines = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for a, b, c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    return None

player = 'X'
while True:
    display()
    pos = int(input(f"Player {player}, enter position (1-9): ")) - 1
    if pos < 0 or pos > 8 or board[pos] != ' ':
        print("Invalid position!")
        continue
    board[pos] = player
    if check_winner():
        display()
        print("Winner:", player)
        break
    if ' ' not in board:
        display()
        print("Game Draw!")
        break
    player = 'O' if player == 'X' else 'X'
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Example terminal session (one possible game):\nPlayer X, enter position (1-9): 1\nPlayer O, enter position (1
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 13: Minimax Algorithm for Gaming

Aim: To implement Minimax to select the best possible value in a two-player game tree.

Algorithm / Procedure

1. Create a game tree with terminal scores.
2. Treat the root as the maximizing player.
3. At maximizing levels, select the maximum value.
4. At minimizing levels, select the minimum value.
5. Return the best value at the root.

Code (screenshot-style)

PYTHON CODE

```
def minimax(tree, depth, maximizing):
    if isinstance(tree, int):
        return tree
    values = [
        minimax(child, depth + 1, not maximizing)
        for child in tree
    ]
    if maximizing:
        return max(values)
    else:
        return min(values)

tree = [
    [3, 5],
    [2, 9]
]
best_value = minimax(tree, 0, True)
print("Best Value:", best_value)
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Best Value: 3
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 14: Alpha-Beta Pruning

Aim: To implement Alpha-Beta Pruning to optimize Minimax search by eliminating unnecessary branches.

Algorithm / Procedure

1. Initialize alpha to negative infinity and beta to positive infinity.
2. Evaluate the game tree using Minimax.
3. At maximizing levels, update alpha.
4. At minimizing levels, update beta.
5. Stop exploring a branch when alpha is greater than or equal to beta.
6. Display the best value.

Code (screenshot-style)

PYTHON CODE

```
def alphabeta(tree, alpha, beta, maximizing):
    if isinstance(tree, int):
        return tree
    if maximizing:
        value = float('-inf')
        for child in tree:
            value = max(value, alphabeta(child, alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for child in tree:
            value = min(value, alphabeta(child, alpha, beta, True))
            beta = min(beta, value)
            if alpha >= beta:
                break
        return value

tree = [
    [3, 5],
    [2, 9]
]
result = alphabeta(tree, float('-inf'), float('inf'), True)
print("Best Value:", result)
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Best Value: 3
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 15: Decision Tree

Aim: To implement a Decision Tree classifier and predict a class from input data.

Algorithm / Procedure

1. Import the required machine-learning library.
2. Load the Iris dataset.
3. Separate input features and target labels.
4. Create and train a Decision Tree classifier.
5. Predict the class of a sample.
6. Display the prediction and training accuracy.

Code (screenshot-style)

PYTHON CODE

```
# Install if needed: pip install scikit-learn
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = model.predict(sample)

print("Predicted Class:", iris.target_names[prediction[0]])
print("Training Accuracy:", model.score(X, y))
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Predicted Class: setosa\nTraining Accuracy: 1.0\n\nNote: Training accuracy is not performance on unseen data.
```

Result: The program demonstrates the stated AI algorithm or technique.

Experiment 16: Feed-Forward Neural Network

Aim: To demonstrate how input data passes through a hidden layer to produce an output in a Feed-Forward Neural Network.

Algorithm / Procedure

1. Initialize the input, weights and biases.
2. Multiply the input by the weights.
3. Add biases and apply the sigmoid activation function.
4. Pass the hidden-layer output to the output layer.
5. Apply the activation function again.
6. Display the final output.

Code (screenshot-style)

PYTHON CODE

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

inputs = np.array([[1.0, 0.5]])

w1 = np.array([[0.2, 0.4],
               [0.3, 0.1]])
b1 = np.array([[0.1, 0.1]])
w2 = np.array([[0.5],
               [0.6]])
b2 = np.array([[0.2]])

hidden = sigmoid(np.dot(inputs, w1) + b1)
output = sigmoid(np.dot(hidden, w2) + b2)

print("Network Output:", output[0][0])
```

Output (screenshot-style)

TERMINAL OUTPUT

```
Network Output: 0.708 (approximately)\n\nNote: Fixed weights are used; the network is not trained.
```

Result: The program demonstrates the stated AI algorithm or technique.